Accueil du niveau

TYPES DE DONNÉES ET STRUCTURES

Les briques de construction de Python

TYPES PYTHON

1 / 10

Accueil du niveau

TYPES NUMÉRIQUES

int et float, les nombres de Python

```
age = 25                # int
pi = 3.14159            # float
grand = 10 ** 100       # int sans limite !

type(age)     # <class 'int'>
type(pi)      # <class 'float'>

# Conversion automatique
resultat = age + pi     # 28.14159 → float
int(3.7)                # 3 (troncature)
float(42)               # 42.0
```

INT · FLOAT

2 / 10

Accueil du niveau

CHAÎNES DE CARACTÈRES

str, le texte en Python

```
nom = "Alice"
prenom = 'Bob'
phrase = "Bonjour le monde"

# f-strings : injection de variables
age = 30
msg = f"{nom} a {age} ans"
# "Alice a 30 ans"

# Méthodes courantes
"python".upper()        # "PYTHON"
"  salut  ".strip()     # "salut"
"a,b,c".split(",")      # ["a", "b", "c"]
len(phrase)             # 16
```

CHAÎNE INDEXÉE

[0]

[1]

[2]

[3]

Accueil du niveau

3 / 10

Accueil du niveau

BOOLÉENS ET NONE

Vrai, faux et l'absence de valeur

```
actif = True
type(actif)        # <class 'bool'>

# Comparaisons → booléens
5 > 3              # True
10 == 11           # False
"a" in "abc"       # True

# None : absence de valeur
resultat = None
type(resultat)     # <class 'NoneType'>
resultat is None   # True (pas ==)
```

TRUE VS FALSE

```
True
```

VS

```
False
```

4 / 10

Accueil du niveau

LES LISTES

La boîte à compartiments de Python

```
fruits = ["pomme", "banane", "cerise"]
notes = [14, 17, 12, 19, 8]
mixte = [1, "hello", True, 3.14]

# Modifier sur place (mutable)
fruits.append("mangue")     # ajoute à la fin
fruits.remove("banane")     # retire par
valeur
notes.sort()                # [8, 12, 14, 17,
19]

len(fruits)    # 3
"pomme" in fruits  # True
```

[1, 2, 3, ...]

→

→

Accueil du niveau

5 / 10

Accueil du niveau

INDEXATION ET SLICING

Accéder aux éléments avec précision

```
lettres = ["a", "b", "c", "d", "e"]

# Indexation
lettres[0]       # "a" (premier)
lettres[-1]      # "e" (dernier)
lettres[-2]      # "d" (avant-dernier)

# Slicing [début:fin:pas]
lettres[1:3]     # ["b", "c"]
lettres[:3]      # ["a", "b", "c"]
lettres[2:]      # ["c", "d", "e"]
lettres[::2]     # ["a", "c", "e"]
```

LST[1:3]

[0]

[1]

[2]

[3]

Accueil du niveau

LES TUPLES

La boîte scellée de Python

```
# Tuple : immuable
coordonnees = (48.8566, 2.3522)  # Paris
couleur_rgb = (255, 128, 0)
jours = ("lun", "mar", "mer", "jeu", "ven")

coordonnees[0]    # 48.8566
len(jours)        # 5

# Impossible de modifier !
# coordonnees[0] = 0  → TypeError

# Déballage (unpacking)
lat, lon = coordonnees
print(f"{lat}°N, {lon}°E")
```

MUTABLE VS IMMUTABLE

Mutable

VS

Immutable

Accueil du niveau

LES DICTIONNAIRES

L'annuaire de Python

```
etudiant = {
    "nom": "Alice",
    "age": 22,
    "notes": [14, 17, 12]
}

etudiant["nom"]           # "Alice"
etudiant["moyenne"] = 14.3  # ajouter
etudiant.get("ville", "?")  # "?" (défaut)

# Parcourir
for cle, val in etudiant.items():
    print(f"{cle} → {val}")

etudiant.keys()    # les clés
etudiant.values()  # les valeurs
```

CLÉ → VALEUR

```
"clé_1" →valeur
"clé_2" →valeur
"clé_3" →valeur
```

8 / 10

Accueil du niveau

Accueil du niveau

LES ENSEMBLES

L'ensemble mathématique en Python

```
fruits = {"pomme", "banane", "cerise"}
nombres = {1, 2, 3, 2, 1}  # → {1, 2, 3}

fruits.add("mangue")
fruits.add("pomme")   # ignoré (déjà présent)

# Éliminer les doublons
liste = [1, 2, 2, 3, 3, 3]
uniques = list(set(liste))  # [1, 2, 3]

# Opérations ensemblistes
a = {1, 2, 3}
b = {2, 3, 4}
a | b   # {1, 2, 3, 4} union
a & b   # {2, 3} intersection
a - b   # {1} différence
```

UNIQUE ∩ ∪

9 / 13

Accueil du niveau

Accueil du niveau

ENCODAGE UNICODE

Gérer proprement accents et caractères spéciaux

```
texte = "café ☕"
raw = texte.encode("utf-8")
print(raw)

decoded = raw.decode("utf-8")
assert decoded == texte
```

- UTF-8 = format recommandé en pratique
- Préciser l'encodage en lecture/écriture fichier
- Éviter les erreurs de conversion silencieuses

10 / 13

Accueil du niveau

REGEX DE BASE

Valider et extraire des motifs texte

```
import re

email = "contact@formation-pro.fr"
ok = re.match(r"^[\\w\\.-]+@[\\w\\.-]+\\.[a-z]{2,}$", email)
print(bool(ok))

texte = "Commande A-2026-015 validée"
ids = re.findall(r"[A-Z]-\\d{4}-\\d{3}", texte)
print(ids)
```

- `match` pour validation
- `findall` pour extraction
- Commencer simple, tester souvent

Accueil du niveau

PRÉCISION DES FLOTTANTS

Comprendre les limites de représentation

```
import math

print(0.1 + 0.2)                # 0.30000000000000004
print((0.1 + 0.2) == 0.3)       # False
print(math.isclose(0.1 + 0.2, 0.3))  # True
```

- Comparer des floats avec tolérance
- Éviter l'égalité stricte sur décimaux
- Calcul monétaire: préférer `decimal`

12 / 13

Accueil du niveau

RÉCAPITULATIF

Vos briques sont prêtes

- ✓ int / float : nombres sans limite et décimaux
- ✓ str : texte immuable avec f-strings
- ✓ bool / None : interrupteurs et absence
- ✓ list : boîte à compartiments (mutable)
- ✓ tuple : boîte scellée (immuable)
- ✓ dict : annuaire clé-valeur
- ✓ set : ensemble sans doublons

13 / 13

Speaker notes